Cyprus University of Technology

CSE 226
Algorithms and Complexity

Lecture 9
Greedy Algorithms (Part 2)
Άπληστοι Αλγόριθμοι (Μέρος 2)

Herodotos Herodotou

Contains edited slides from T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, D. Luebke, and M. Christodoulakis

1

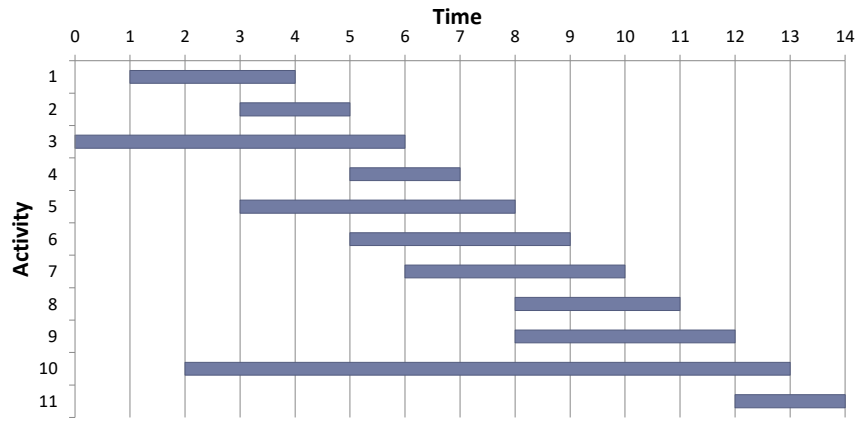
Recall: Greedy Algorithms

- **Greedy strategy** (**άπληστη στρατηγική**)
  - Make the choice that seems best right now!
- What did we do for activity selection?
  1. Determined the **optimal substructure** (**βέλτιστη υποδομή**)
  2. Developed a **recursive solution** (**αναδρομική λύση**)
  3. Proved that at any stage of recursion, one of the optimal choices was the **greedy choice** (**άπληστη επιλογή**). Therefore, it was always safe to make the greedy choice
  4. Showed that all but one of the subproblems resulting from the greedy choice are empty
  5. Developed a recursive greedy algorithm
  6. Converted it to an iterative algorithm

2

Recall: Activity Selection Problem

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	8	9	10	11	12	13	14



3

Activity Selection Problem Revisited

- Develop the substructure with an eye toward
  - making the greedy choice
  - leaving just one subproblem
- For activity selection, we showed that the greedy choice implied that in $S_{ij} = \{a_k \in S : f_i \leq s_k < f_k \leq s_j\}$, only i varied, and j was fixed at $n + 1$
- We could have started out with a greedy algorithm in mind:
  - Define $S_i = \{a_k \in S : f_i \leq s_k\}$
  - Then show that the greedy choice (first a_m to finish in S_i) combined with optimal solution to $S_m \Rightarrow$ optimal solution to S_i

4

Greedy Algorithm - Steps

1. Cast the optimization problem as one in which we make *a choice* (*επιλογή*) and are left with *one subproblem* (*υποπρόβλημα*) to solve
2. Prove that there is always an *optimal solution* (*βέλτιστη λύση*) to the original problem that makes the *greedy choice* (*άπληστη επιλογή*), so that the greedy choice is always safe
3. Show that greedy choice and optimal solution to subproblem ⇒ optimal solution to the problem

- Key ingredients for greedy strategy:
  1. **greedy-choice property** (**ιδιότητα άπληστης επιλογής**)
  2. **optimal substructure** (**βέλτιστη υποδομή**)

Greedy-choice Property

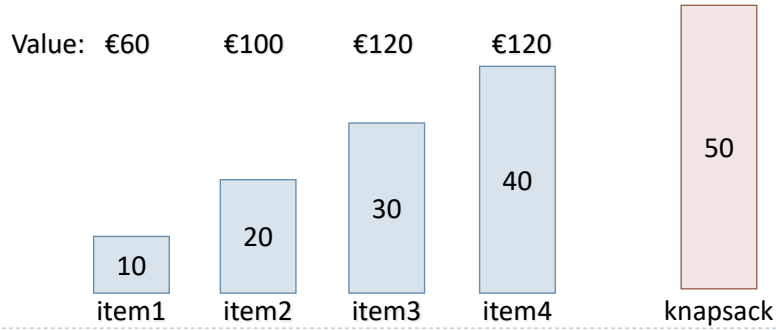
- A globally optimal solution can be arrived at by making a locally optimal (greedy) choice

Greedy Algorithm	Dynamic Programming
Make a (greedy) choice at each step	Make a choice at each step
Make the choice *before* solving the subproblems	Choice depends on knowing optimal solutions to subproblems. Solve subproblems *first*
Solve *top-down*	Solve *bottom-up*

- Steps to show the greedy-choice property:
  1. Look at a globally optimal solution
  2. If it includes the greedy choice, done
  3. Else, modify it to include the greedy choice, yielding another solution that's just as good

Example: Fractional Knapsack Problem

- **Input:** a collection of n items $G = \{g_1, g_2, \dots, g_n\}$, where each item g_i is worth v_i euros and weights w_i kgs
- **Output:** the *fractions* x_i of the items to take from G, that have total weight at most W kgs and their total value V is the largest possible (maximize $\mathrm{V} = \sum_i x_i v_i$ such that $\sum_i x_i w_i \leq W$)



Greedy Algorithm – Step 1

- Construct the **greedy optimal solution**
  1. Sort the items by their value per kg ratio v_i/w_i so that g_1 is the item with the greatest value per kg, g_2 the item with the next greatest value per kg, and so on
  2. Take as much as possible of g_1 (*greedy choice*)
  3. If there is more space in the bag, take as much as possible of g_2
  4. Continue in this fashion until there is no more space
- Notice: In the solution above, we make the greedy choice (the one with most value from the current options) and are left with one subproblem to solve (which one?)

Greedy Algorithm – Step 2

- Prove the **greedy-choice property**, i.e., that the greedy choice leads to (or is contained within) an optimal solution
- There are two cases:

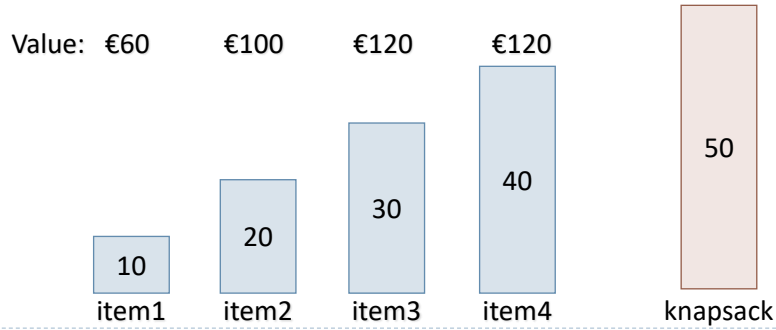
1. $w_1 \leq W$ (i.e., the whole of g_1 fits in the bag)
   - Assume the optimal solution contains $w' < w_1$ kgs of g_1
   - Then, remove any items of weight $w_1 - w'$ Kg from the optimal solution and replace them with $w_1 - w'$ kgs of g_1
   - Observe that the new solution will have the same weight but larger value! Contradiction (since we assumed it was optimal)
2. $w_1 > W$ (the greedy choice fills the bag with a fraction of g_1)
   - Assume the optimal solution contains $w' < W$ kgs of g_1
   - Then, replace those $W - w'$ kgs of items that are not g_1 with $W - w'$ kgs of g_1 and the value of the bag increases. Contradiction!

Greedy Algorithm – Step 3

- Prove the **optimal substructure property**, i.e., the optimal solution to the problem contains the optimal solution to the subproblem that remains after the greedy choice is made
- Notice that after making the greedy choice we are left with one subproblem of size $W - w_1$
  - Unless $w_1 > W$, in which case there is no subproblem to be solved
- **Proof by contradiction** (Εις άτοπον απαγωγή):
  - Assume that the optimal solution $S_n = \{x_1 .. x_n\}$ to the problem of size W does not contain the optimal solution $S_{n-1} = \{y_2 .. y_n\}$ to the subproblem $W - w_1$, where selection can now be made from the remaining $n - 1$ items
  - Then, we can replace the sub-optimal solution of the subproblem with an optimal one, and obtain larger value V'
  - $V = x_1 v_1 + \sum_{i=2}^{n} x_i v_i < x_1 v_1 + \sum_{i=2}^{n} y_i v_i = V'$ → Contradiction!

Exercise

- The 0-1 knapsack problem is similar to the fractional one, but now fractions of items are **not** allowed: either you take item g_i as a whole or you leave it. Prove with an example that the greedy choice does not work in this problem



	Divide & Conquer	Dynamic Progr.	Greedy Algorithm
View problem as collection of subproblems	✓	✓	✓
"Recursive" nature	✓	✓	✓
Dependence of subproblems	Independent	Overlapping	Typically sequential dependence
Number of subproblems	Depends on partitioning factors	Typically small	
Preprocessing			Typically sort
Characteristic running time	Typically log function of n	Depends on # and difficulty of subproblems	Often dominated by $n \log n$ sort
Primarily for optimization problems		✓	✓
Optimal substructure		✓	✓
Greedy-choice property			✓